

Documentación Técnica

1. Selección del Formato de Configuración

Para almacenar las preferencias del usuario, se ha optado por el formato **JSON**. Se presenta una comparativa técnica frente a una alternativa común, los archivos **CSV**, para justificar la decisión arquitectónica.

Criterio Técnico	Elegido: JSON	Descartado: CSV
Estructura	Jerárquica (clave-valor, diccionarios, arreglos) Permite datos anidados	Plana y tabular (filas y columnas). Similar a una hoja de Excel.
Tipos de datos	Soporta tipos nativos. (cadenas de texto, números, booleanos, valores nulos)	Todo es texto plano. Los números o booleanos se deben convertir manualmente en el código
Legibilidad	Alta. El formato con llaves deja claro a que propiedad pertenece cada valor.	Baja si hay muchas columnas, además es difícil saber que dato corresponde a que campo pertenece.
Caso de uso ideal	Archivos de configuración, respuesta de APIs y transferencia de datos estructurados.	Exportación de bases de datos, reportes financieros y análisis de datos masivos.

1. Implementación de Escritura Segura y Respaldo

Para evitar la corrupción de la configuración por apagones o interrupciones durante el proceso de guardado, se implementó un patrón de escritura atómica complementado con respaldos automáticos.

Flujo de Guardado (Código Relevante)

1. Se valida la configuración antes de procesar archivos.
2. Se escribe sobre un archivo temporal (`config.tmp`) usando `ensure_ascii=False`.
3. Se fuerzan los búferes al disco mediante `f.flush()` y `os.fsync(f.fileno())`.
4. Si existe una versión previa en disco, se respalda copiándola a `config.bak` mediante `shutil.copy2`.
5. Finalmente, se renombra atómicamente el temporal a `config.json` usando `os.replace()`.

```
def save_config(config):
    # [...] Validación inicial
    try:
        with open(TEMP_FILE, "w", encoding="utf-8") as f:
            json.dump(config_validado, f, ensure_ascii=False, indent=4)
            f.flush()
            os.fsync(f.fileno()) # Escritura forzada a disco duro

        if os.path.exists(CONFIG_FILE):
            try:
                shutil.copy2(CONFIG_FILE, BACKUP_FILE) # Creación de respaldo
            except OSError:
                pass

        os.replace(TEMP_FILE, CONFIG_FILE) # Reemplazo atómico
        return ConfigResult(config_validado, "ok")

    except PermissionError:
        _limpiar_temporal()
        return ConfigResult(..., "permission_denied", ...)
```

2. Manejo de Errores y Evidencias

El script `manejoErrores.py` permite probar sistemáticamente las tres fallas exigidas durante la inicialización o lectura del archivo. A continuación, el detalle y la evidencia arrojada por la consola (logs):

Caso A: Archivo Ausente

Manejo: Se comprueba `if not os.path.exists(CONFIG_FILE)` en `load_config()`. Si falta, el sistema no colapsa; devuelve un diccionario clonado `DEFAULT_CONFIG.copy()` con estado `"not_found"`.

```
===== Caso 1:
archivo de configuracion ausente
===== estado =
not_found Mensaje = No existe un archivo de configuracion previo. datos =
{'nombre_usuario': 'Usuario', 'tema_interfaz': 'claro', ...} Se cargaron valores por
defecto
```

Caso B: Archivo Corrupto (JSON Inválido)

Manejo: Se captura la excepción `json.JSONDecodeError` al intentar leer el archivo principal. Se invoca a `_intentar_cargar_backup()`. Si hay un archivo `.bak` válido, se recupera el estado y devuelve `"corrupt_recovered"`. De lo contrario, se devuelven los valores por defecto con estado `"corrupt"`.

```
===== Caso 2:
archivo corrupto // formato invalido
===== Estado =
corrupt Mensaje = El archivo estaba dañado (Expecting value: line 1 column 1 (char
0)) y no habia ningunrespaldo utilizable. Datos = {'nombre_usuario': 'Usuario',
'tema_interfaz': 'claro', ...} Archivo corrupto detectado
```

Caso C: Archivo Sin Permisos (Acceso Denegado)

Manejo: Se usa un bloque `try...except PermissionError` tanto en lectura como escritura. Si la lectura falla, el sistema provee una configuración por defecto temporal en RAM, permitiendo a la app ejecutarse, y el estado retornado notifica `"permission_denied"` para levantar alertas visuales en UI.

```
===== Caso 3:
archivo sin permisos de lectura
===== estado =
permission_denied Mensaje = Sin permisos de lectura sobre 'config.json'. Datos =
{'nombre_usuario': 'Usuario', 'tema_interfaz': 'claro', ...} Sin permisos de lectura
```

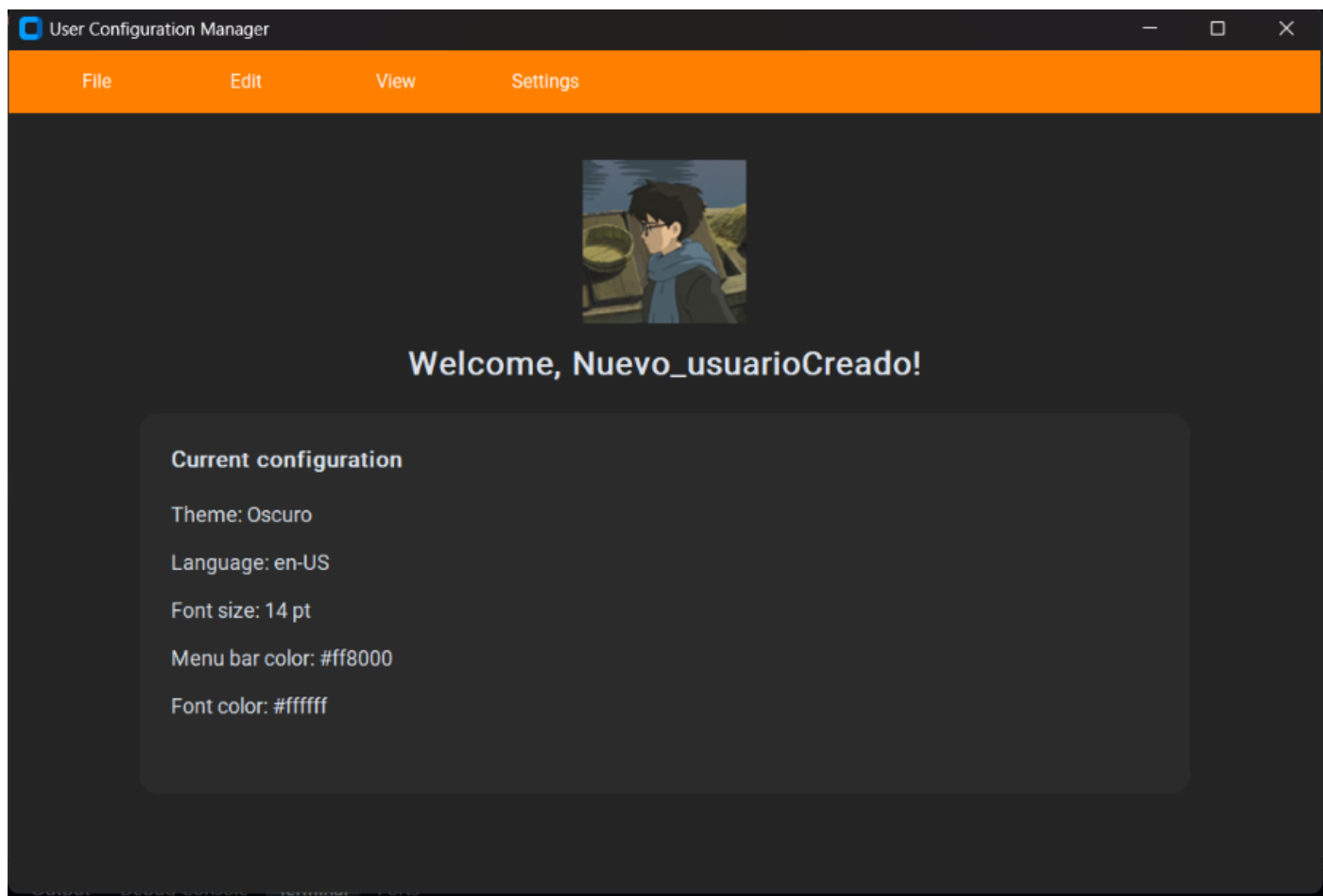
3. Ejemplo del Archivo Generado (Manejo de Codificación)

Para asegurar la compatibilidad con el idioma (ej: Español) y evitar escapes que dañan la lectura, en el guardado se empleó el argumento `ensure_ascii=False` junto al modo `encoding="utf-8"`.

A continuación, se muestra un ejemplo real del archivo `config.json` evidenciando la persistencia correcta de tildes y caracteres especiales (ñ, á, í).

```
{
  "nombre_usuario": "José Ibañez",
  "tema_interfaz": "oscuro",
  "idioma": "es-ES",
  "tamaño_fuente": 14,
  "color_menu": "#1A2B3C",
  "color_letra": "#FFFFFF",
  "foto_perfil": "C:\\Usuarios\\José\\Imágenes\\diseño año nuevo.png"
}
```

4. Capturas de pantalla del funcionamiento de la aplicación



Pantalla principal (usuario usando modo oscuro, mostrando la información abajo, en idioma ingles)

Simulated feature X



This option is not available

OK

Mensaje de prueba a los botones que no son funcionales (archivo, editar, vista)

Settings — Configuration X

Username:

Nuevo_usuarioCreado

Interface theme:

☐ Claro / Light ☒ Oscuro / Dark

Language:

en-US

Font size:

14 + -

Menu bar color:

Choose color...

Font color:

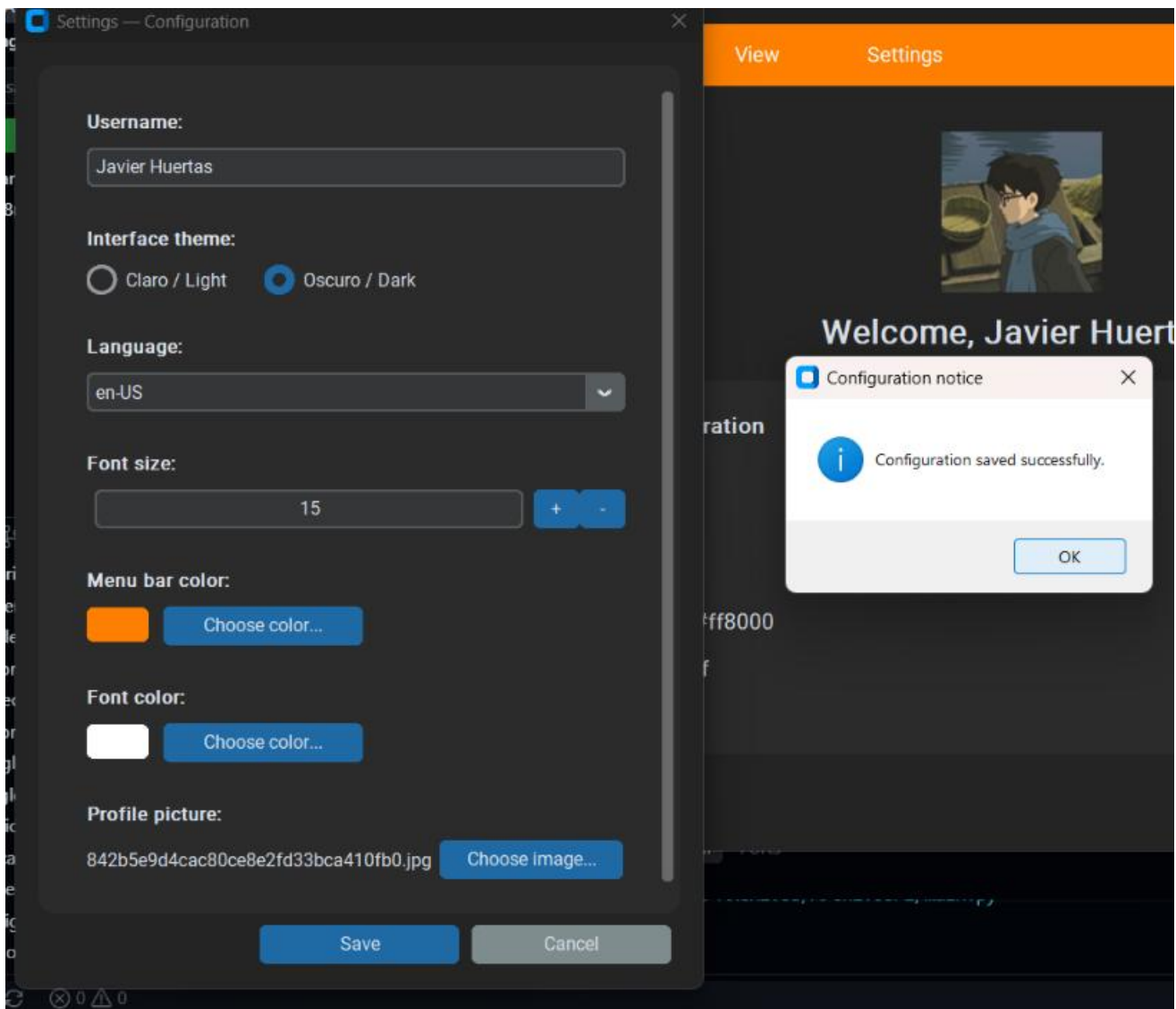
Choose color...

Profile picture:

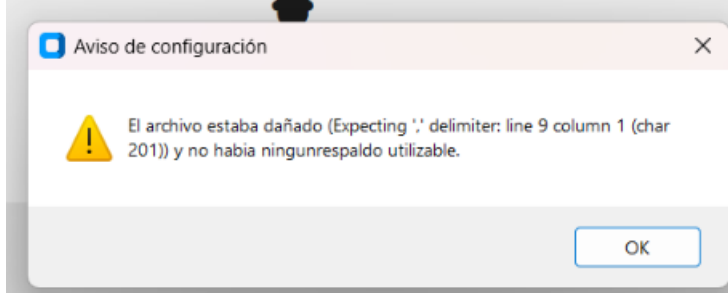
842b5e9d4cac80ce8e2fd33bca410fb0.jpg Choose image...

Save Cancel

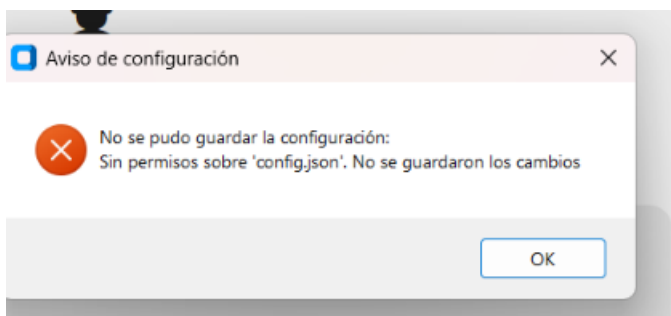
Ventana de settings



Cambios al nombre de usuario y tamaño de letra junto con el mensaje de configuración guardada



aviso de error si el archivo JSON esta dañado, vuelve todo a defecto



Error si el usuario no tiene permisos